

▼ Data Preprocessing

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import nltk
import re
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from wordcloud import WordCloud
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

Features Table

```
from IPython.display import Markdown

table_text = """
| Feature Name | Description |
|-----|-----|
| Hotel_Address | Address of hotel |
| Review_Date | Date when reviewer posted the corresponding review |
| Average_Score | Average score of the hotel, based on latest comments in the last year |
| Hotel_Name | Name of the hotel |
| Reviewer_Nationality | Nationality of the reviewer |
| Negative_Review | Negative review text. 'No Negative' if none provided |
| Review_Total_Negative_Word_Counts | Total number of words in the negative review |
| Positive_Review | Positive review text. 'No Positive' if none provided |
| Review_Total_Positive_Word_Counts | Total number of words in the positive review |
| Reviewer_Score | Score the reviewer gave based on their experience |
| Total_Number_of_Reviews_Reviewer_Has_Given | Number of reviews the reviewer has given in the past |
| Total_Number_of_Reviews | Total number of valid reviews for the hotel |
| Tags | Tags the reviewer gave to the hotel |
| days_since_review | Duration between the review date and scrape date |
| Additional_Number_of_Scoring | Number of scores given without a written review |
| lat | Latitude of the hotel |
| lng | Longitude of the hotel |
"""
```

```
Markdown(table_text)
```

Feature Name	Description
Hotel_Address	Address of hotel
Review_Date	Date when reviewer posted the corresponding review
Average_Score	Average score of the hotel, based on latest comments in the last year
Hotel_Name	Name of the hotel
Reviewer_Nationality	Nationality of the reviewer
Negative_Review	Negative review text. 'No Negative' if none provided
Review_Total_Negative_Word_Counts	Total number of words in the negative review
Positive_Review	Positive review text. 'No Positive' if none provided
Review_Total_Positive_Word_Counts	Total number of words in the positive review
Reviewer_Score	Score the reviewer gave based on their experience
Total_Number_of_Reviews_Reviewer_Has_Given	Number of reviews the reviewer has given in the past
Total_Number_of_Reviews	Total number of valid reviews for the hotel
Tags	Tags the reviewer gave to the hotel
days_since_review	Duration between the review date and scrape date
Additional_Number_of_Scoring	Number of scores given without a written review
lat	Latitude of the hotel
lng	Longitude of the hotel

```
df = pd.read_csv('/content/Hotel_Reviews.csv')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 515738 entries, 0 to 515737
Data columns (total 17 columns):
 #   Column                                          Non-Null Count  Dtype
---  -
 0   Hotel_Address                                515738 non-null  object
 1   Additional_Number_of_Scoring                 515738 non-null  int64
 2   Review_Date                                  515738 non-null  object
 3   Average_Score                                515738 non-null  float64
 4   Hotel_Name                                    515738 non-null  object
 5   Reviewer_Nationality                        515738 non-null  object
 6   Negative_Review                             515738 non-null  object
 7   Review_Total_Negative_Word_Counts           515738 non-null  int64
 8   Total_Number_of_Reviews                     515738 non-null  int64
 9   Positive_Review                             515738 non-null  object
10   Review_Total_Positive_Word_Counts           515738 non-null  int64
11   Total_Number_of_Reviews_Reviewer_Has_Given  515738 non-null  int64
12   Reviewer_Score                               515738 non-null  float64
13   Tags                                          515738 non-null  object
14   days_since_review                           515738 non-null  object
15   lat                                           512470 non-null  float64
16   lng                                           512470 non-null  float64
dtypes: float64(4), int64(5), object(8)
memory usage: 66.9+ MB
```

```
df.head(10)
```

	Hotel_Address	Additional_Number_of_Scoring	Review_Date	Average_Score	Hotel_Name	Reviewer_Nationality	Ne
0	Gravesandestraat 55 Oost 1092 AA Amsterdam ...	194	8/3/2017	7.7	Hotel Arena	Russia	Is
1	Gravesandestraat 55 Oost 1092 AA Amsterdam ...	194	8/3/2017	7.7	Hotel Arena	Ireland	
2	Gravesandestraat 55 Oost 1092 AA Amsterdam ...	194	7/31/2017	7.7	Hotel Arena	Australia	Rc
3	Gravesandestraat 55 Oost 1092 AA Amsterdam ...	194	7/31/2017	7.7	Hotel Arena	United Kingdom	Ma
4	Gravesandestraat 55 Oost 1092 AA Amsterdam ...	194	7/24/2017	7.7	Hotel Arena	New Zealand	Yc w
5	Gravesandestraat 55 Oost 1092 AA Amsterdam ...	194	7/24/2017	7.7	Hotel Arena	Poland	f
6	Gravesandestraat 55 Oost 1092 AA Amsterdam ...	194	7/17/2017	7.7	Hotel Arena	United Kingdom	
7	Gravesandestraat 55 Oost 1092 AA Amsterdam ...	194	7/17/2017	7.7	Hotel Arena	United Kingdom	Ap
8	Gravesandestraat 55 Oost 1092 AA Amsterdam ...	194	7/9/2017	7.7	Hotel Arena	Belgium	p
9	Gravesandestraat 55 Oost 1092 AA Amsterdam ...	194	7/8/2017	7.7	Hotel Arena	Norway	

▼ Data Cleaning

```
df['Positive_Review'] = df['Positive_Review'].replace('No Positive', '')
df['Negative_Review'] = df['Negative_Review'].replace('No Negative', '')
df['overall_review'] = df['Positive_Review'] + ' ' + df['Negative_Review']

df['days_since_review'] = df['days_since_review'].astype(str).str.extract(r'(\d+)').astype(float)
```

```
df['overall_review'].head()
```

```

overall_review
0    Only the park outside of the hotel was beauti...
1    No real complaints the hotel was great great ...
2    Location was good and staff were ok It is cut...
3    Great location in nice surroundings the bar a...
4    Amazing location and building Romantic settin...

```

```
dtype: object
```

```
print(df['overall_review'].iloc[0])
print(df['overall_review'].iloc[1])
```

```

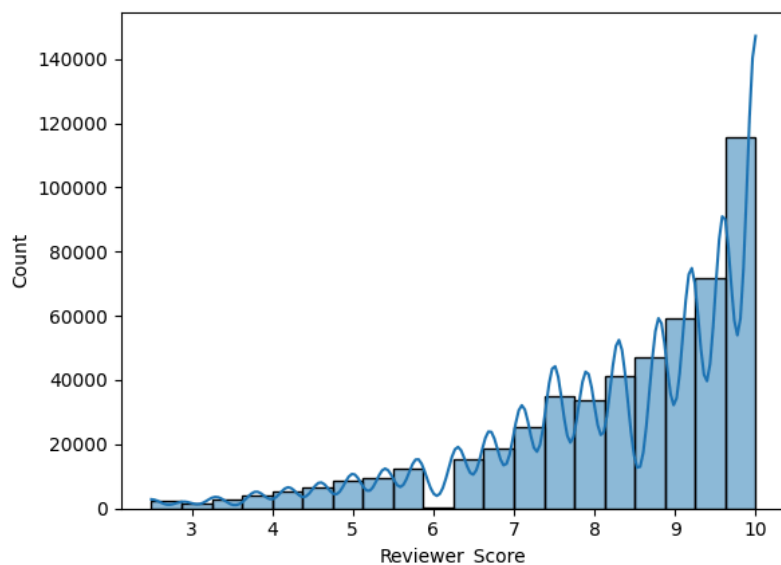
Only the park outside of the hotel was beautiful    I am so angry that i made this post available via all possib
No real complaints the hotel was great great location surroundings rooms amenities and service Two recommendatio

```

EDA

```
sns.histplot(df['Reviewer_Score'], bins=20, kde=True)
```

```
<Axes: xlabel='Reviewer_Score', ylabel='Count'>
```



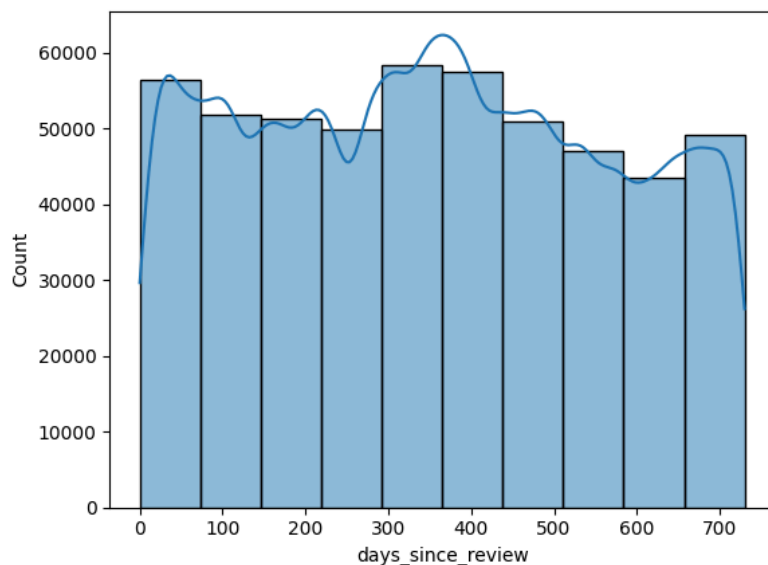
```
#Total_Number_of_Reviews
```

```
#days_since_review
```

```
df['days_since_review']= df['days_since_review'].astype(str).str.extract('(\d+)').astype(float)
```

```
sns.histplot(df['days_since_review'], bins=10, kde=True)
```

<Axes: xlabel='days_since_review', ylabel='Count'>



```
max = df['days_since_review'].max()
print(max)
```

```
min = df['days_since_review'].min()
print(min)
```

```
730.0
0.0
```

```
top_10_hotels = df['Hotel_Name'].value_counts().head(10)
```

```
top_10_hotels = df['Hotel_Name'].value_counts().head(10)
```

```
plt.figure(figsize=(10, 6))
```

```
ax = sns.barplot(x=top_10_hotels.values, y=top_10_hotels.index, palette='Blues_r')
```

```
for i, v in enumerate(top_10_hotels.values):
    ax.text(v + 20, i, str(v), color='black', va='center')
```

```
plt.xlabel("Number of Reviews")
```

```
plt.ylabel("Hotel Name")
```

```
plt.title("Top 10 Most Reviewed Hotels with Labels")
```

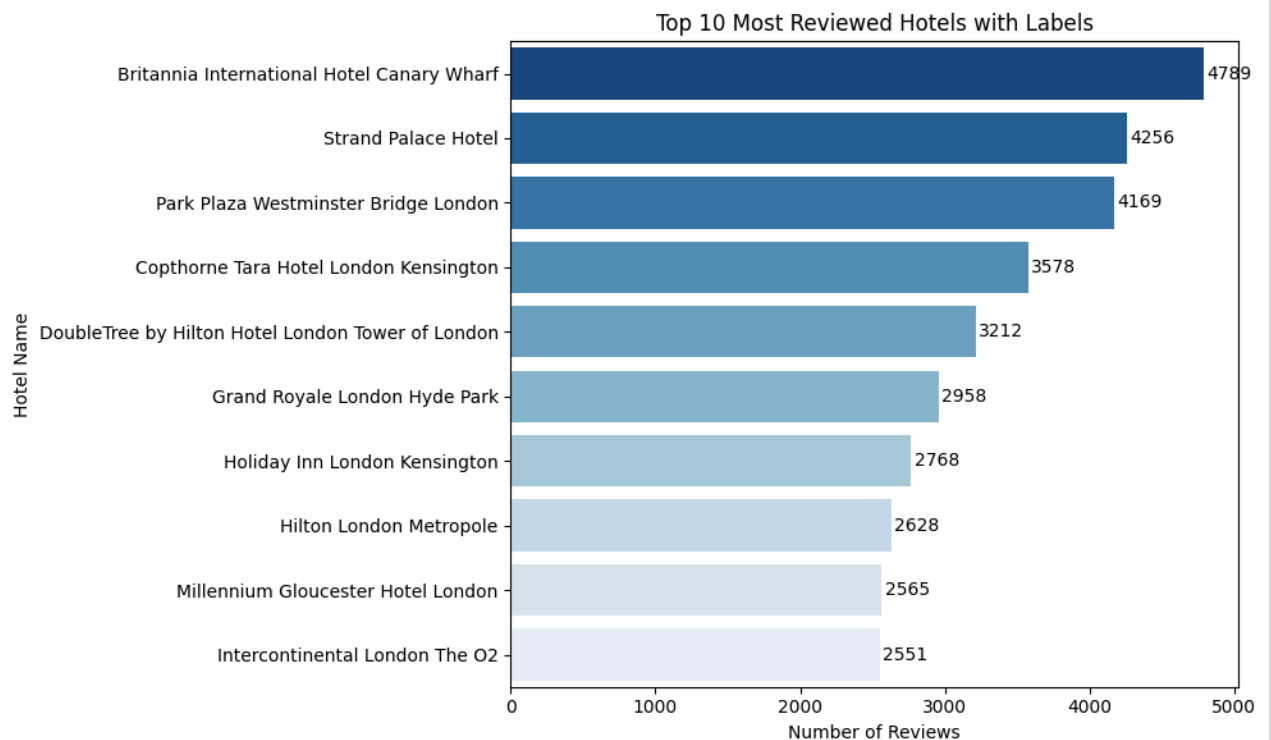
```
plt.tight_layout()
```

```
plt.show()
```

/tmp/ipython-input-3243325288.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable

```
ax = sns.barplot(x=top_10_hotels.values, y=top_10_hotels.index, palette='Blues_r')
```



```
# Bottom 10 hotels by avg review scores.
```

```
bottom_avg_score = df.groupby('Hotel_Name')['Average_Score'].mean().sort_values(ascending=True).head(10)
```

```
plt.figure(figsize=(12,6))
```

```
sns.barplot(x=bottom_avg_score.values, y=bottom_avg_score.index, palette='viridis')
```

```
plt.xlabel("Average Score")
```

```
plt.ylabel("Hotel Name")
```

```
plt.title("Bottom 10 Hotels by Average Review Score")
```

```
ax = plt.gca()
```

```
for i, v in enumerate(bottom_avg_score.values):
```

```
    ax.text(
```

```
        v + 0.05,
```

```
        i,
```

```
        f"{v:.2f}",
```

```
        color='black',
```

```
        va='center'
```

```
    )
```

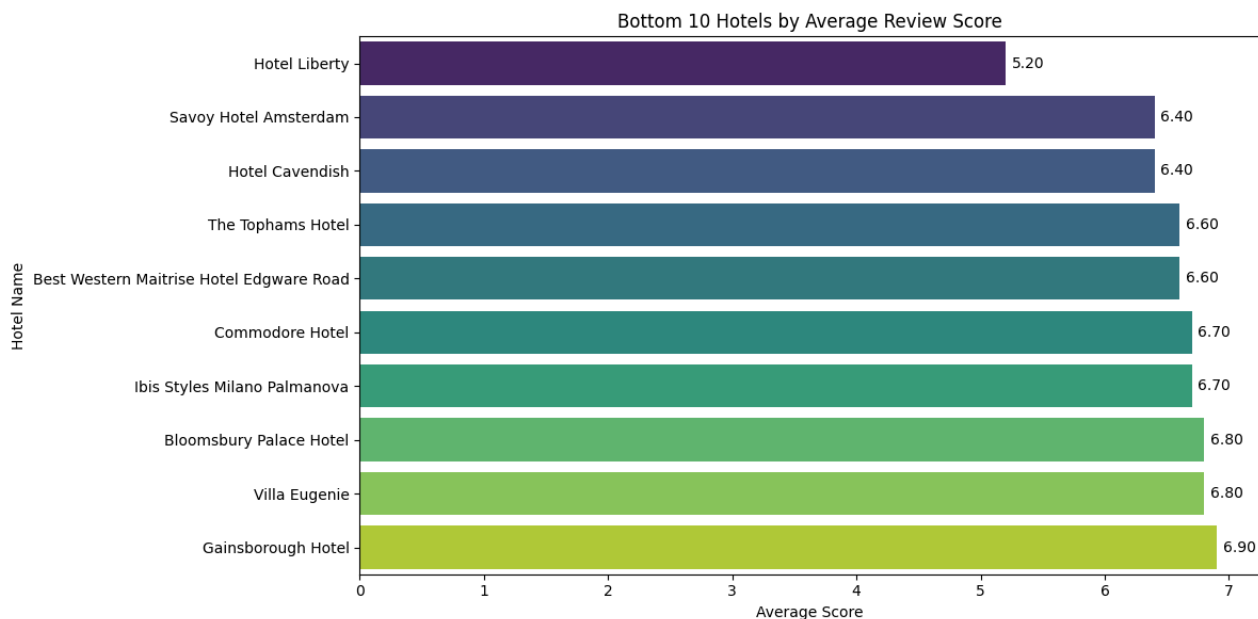
```
plt.tight_layout()
```

```
plt.show()
```

```
/tmp/ipython-input-1049234696.py:5: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable
```

```
sns.barplot(x=bottom_avg_score.values, y=bottom_avg_score.index, palette='viridis')
```



```
df.groupby('Reviewer_Nationality')['Reviewer_Score'].mean().sort_values(ascending=True).head(10)
```

Reviewer_Score	
Reviewer_Nationality	
Montserrat	5.400000
American Samoa	5.650000
Northern Mariana Islands	5.800000
Mali	6.440000
Nepal	6.840000
Afghanistan	6.875000
Gabon	7.085714
Tuvalu	7.100000
Lesotho	7.233333
Mongolia	7.251282

```
dtype: float64
```

```
country_counts = df['Reviewer_Nationality'].value_counts().head(20)
print(country_counts)
```

Reviewer_Nationality	
United Kingdom	245246
United States of America	35437
Australia	21686
Ireland	14827
United Arab Emirates	10235
Saudi Arabia	8951
Netherlands	8772
Switzerland	8678
Germany	7941
Canada	7894
France	7296
Israel	6610
Italy	6114
Belgium	6031
Turkey	5444
Kuwait	4920
Spain	4737
Romania	4552
Russia	3900
South Africa	3821
Name: count, dtype: int64	

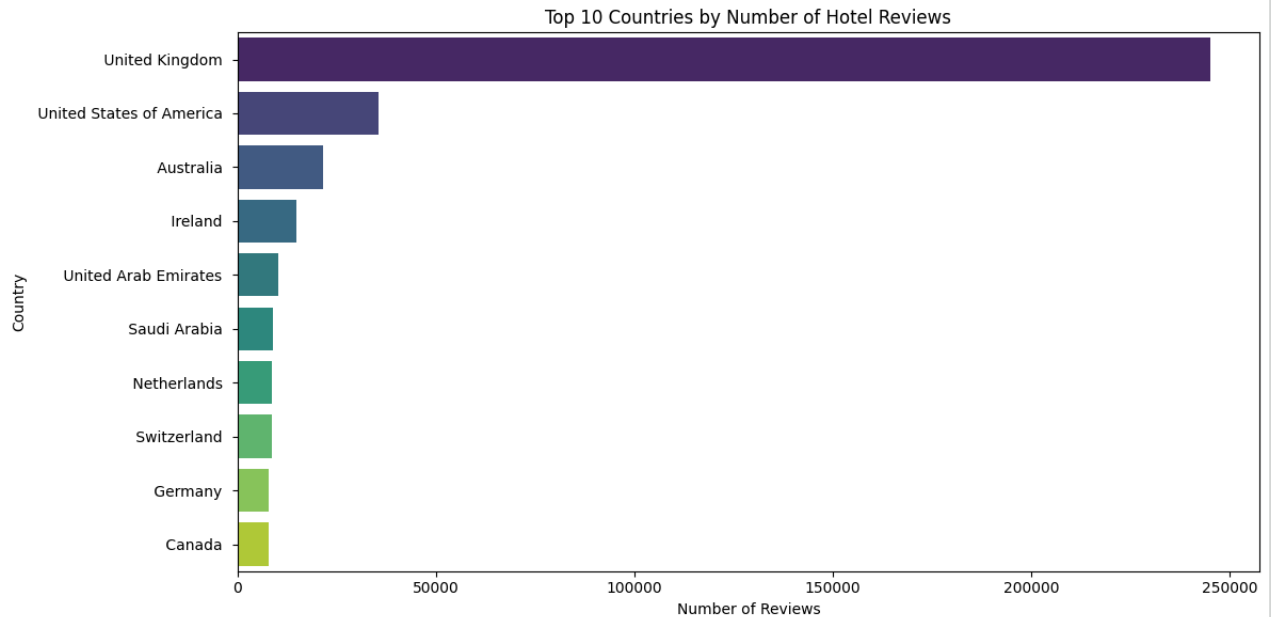
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12,6))
sns.barplot(x=country_counts.head(10).values, y=country_counts.head(10).index, palette='viridis')
plt.title('Top 10 Countries by Number of Hotel Reviews')
plt.xlabel('Number of Reviews')
plt.ylabel('Country')
plt.tight_layout()
plt.show()
```

/tmp/ipython-input-3861498420.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable

```
sns.barplot(x=country_counts.head(10).values, y=country_counts.head(10).index, palette='viridis')
```



```
# See how many hotel players in the datasets
unique_hotels = df['Hotel_Name'].nunique()
print(f"Number of unique hotel names: {unique_hotels}")
```

Number of unique hotel names: 1492

```
hotel_name_counts = df['Hotel_Name'].value_counts()
print(hotel_name_counts.head(10))
```

```
Hotel_Name
Britannia International Hotel Canary Wharf    4789
Strand Palace Hotel                          4256
Park Plaza Westminster Bridge London         4169
Copthorne Tara Hotel London Kensington       3578
DoubleTree by Hilton Hotel London Tower of London 3212
Grand Royale London Hyde Park                2958
Holiday Inn London Kensington                2768
Hilton London Metropole                      2628
Millennium Gloucester Hotel London           2565
Intercontinental London The 02                2551
Name: count, dtype: int64
```

Review Sentiment Classification

```
# 1. classify reviewer score, categorized as 1(positive) if greater than 8 otherwise as 0 (negative)
df['sentiment_label'] = df['Reviewer_Score'].apply(lambda x: 1 if x >= 8 else 0)
```

```
# 2. TF-IDF
from sklearn.feature_extraction.text import TfidfVectorizer
df['overall_review'] = df['overall_review'].astype(str)
```

```
text_data = df['overall_review']
tfidf = TfidfVectorizer(max_features=5000, stop_words='english')
X = tfidf.fit_transform(text_data)
```



```
# 3. Label column
y = df['sentiment_label']
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

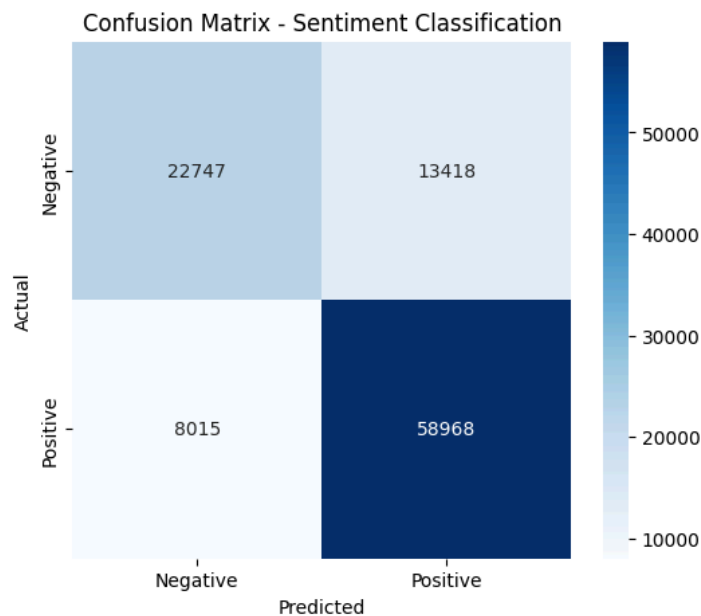
```
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# print classification report
print(classification_report(y_test, y_pred))

# confusion matrix display
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Negative', 'Positive'], yticklabels=['Negative', 'Positive'])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Sentiment Classification")
plt.show()
```

		precision	recall	f1-score	support
	0	0.74	0.63	0.68	36165
	1	0.81	0.88	0.85	66983
accuracy				0.79	103148
macro avg		0.78	0.75	0.76	103148
weighted avg		0.79	0.79	0.79	103148



✓ Data Modeling - Rating Prediction

It is not only predicting or guess a "score", For example, which could be used for quantifying customers' satisfaction even without given particular review_score. Also like in business practice, Airbnb company can leverage such method to assess whether overall review_score is inflated for hosts based on corresponding predicted scores from its' guests' feedbacks.

✓ Random Forest

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Random sampling n=20000
df = pd.read_csv("Hotel_Reviews.csv").sample(n=20000, random_state=42).reset_index(drop=True)

# overall_review
if 'overall_review' not in df.columns:
    pos = df['Positive_Review'].replace('No Positive', '').fillna('')
    neg = df['Negative_Review'].replace('No Negative', '').fillna('')
    df['overall_review'] = (pos.astype(str) + ' ' + neg.astype(str)).str.strip()

# make sure days_since_review as float type
if df['days_since_review'].dtype != 'float':
    df['days_since_review'] = df['days_since_review'].astype(str).str.extract(r'(\d+)').astype(float)

# Identify features and target
features = [
    'Review_Total_Positive_Word_Counts',
    'Review_Total_Negative_Word_Counts',
    'Total_Number_of_Reviews',
    'Total_Number_of_Reviews_Reviewer_Has_Given',
    'Reviewer_Nationality',
    'Tags',
    'Hotel_Name',
    'days_since_review',
    'overall_review'
]
target = 'Reviewer_Score'

# Drop NA records
df = df.dropna(subset=[target])
X = df[features]
y = df[target]

# Classify categorical and numerical features
categorical_features = ['Reviewer_Nationality', 'Tags', 'Hotel_Name']
numerical_features = [
    'Review_Total_Positive_Word_Counts',
    'Review_Total_Negative_Word_Counts',
    'Total_Number_of_Reviews',
    'Total_Number_of_Reviews_Reviewer_Has_Given',
    'days_since_review'
]
```

```

# Preprocess step
preprocessor = ColumnTransformer(
    transformers=[
        ('text', TfidfVectorizer(max_features=5000, stop_words='english'), 'overall_review'),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
        ('num', 'passthrough', numerical_features)
    ],
    remainder='drop'
)

# Build pipeline for streamline the process
pipeline = Pipeline(steps=[
    ('preprocessing', preprocessor),
    ('regressor', RandomForestRegressor(n_estimators=100, random_state=42))
])

# Split test and train datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
pipeline.fit(X_train, y_train)

# Evaluate the result
y_pred = pipeline.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)

# Calculate RMSE by taking the square root of MSE
rmse = mean_squared_error(y_test, y_pred) # Removed squared=False
rmse = np.sqrt(rmse) # Take the square root to get RMSE

print(f"MAE: {mae:.3f} | RMSE: {rmse:.3f}")

```

MAE: 0.901 | RMSE: 1.239

```

import numpy as np
import matplotlib.pyplot as plt

# get feature names from the fitted ColumnTransformer
def get_feature_names(preprocessor, cat_cols, num_cols):
    names = []
    for name, trans, cols in preprocessor.transformers_:
        if name == 'text':
            names.extend([f"TEXT:{w}" for w in trans.get_feature_names_out()])
        elif name == 'cat':
            names.extend(trans.get_feature_names_out(cat_cols))
        elif name == 'num':
            names.extend(num_cols)
    return np.array(names)

# use the already-fitted pipeline components
preproc = pipeline.named_steps['preprocessing']
rf = pipeline.named_steps['regressor']

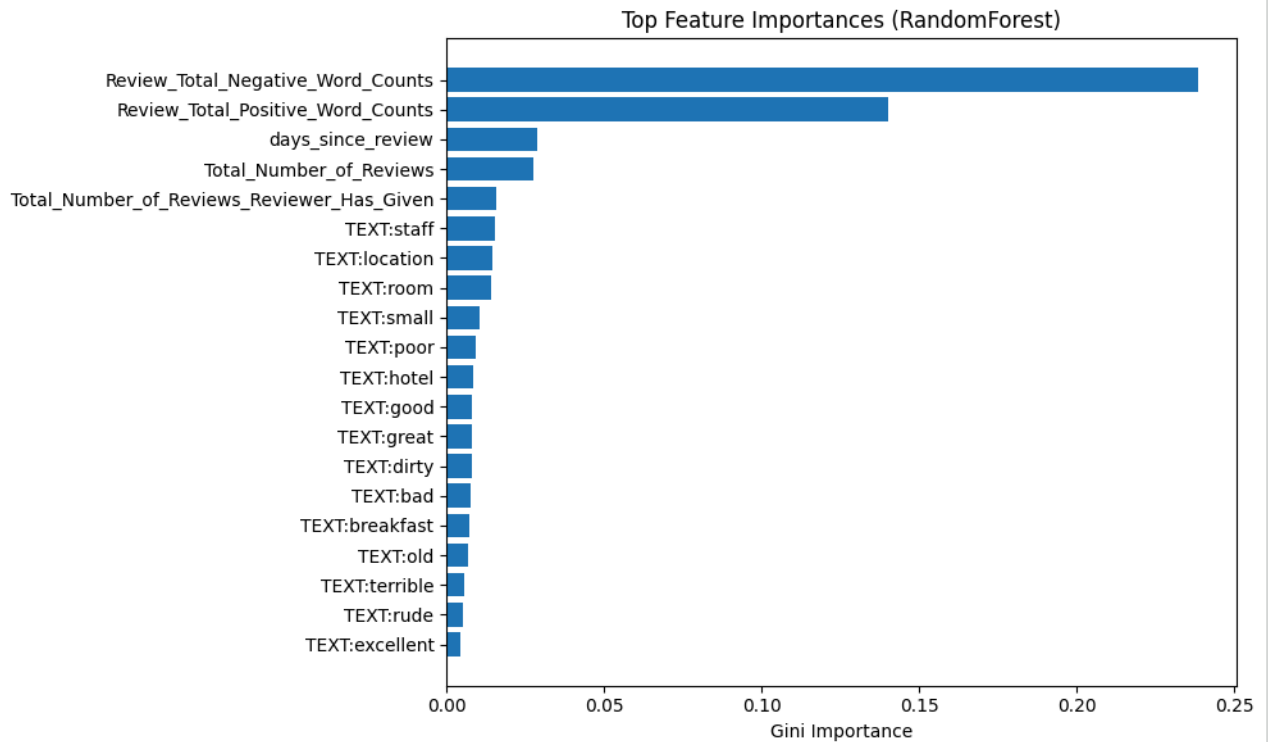
# use transform (NOT fit_transform) to match the model that was already trained
X_train_trans = preproc.transform(X_train)

# get names
feat_names = get_feature_names(preproc, categorical_features, numerical_features)

# feature importances from the already-fitted RandomForest
importances = rf.feature_importances_

# plot top 20
top_n = 20
idx = np.argsort(importances)[-top_n:]
plt.figure(figsize=(10, 6))
plt.barh(range(top_n), importances[idx])
plt.yticks(range(top_n), feat_names[idx])
plt.xlabel("Gini Importance")
plt.title("Top Feature Importances (RandomForest)")
plt.tight_layout()
plt.show()

```



```
# Model testing in one set of new data
```

```
from os import pipe
import pandas as pd
import string
```

```
positive_review = "The hotel was clean and the staff were friendly, great location near the metro."
negative_review = "No Negative"
```

```
# Remove punctuation and count only alphabetic words
```

```
def word_count(text):
    text = text.translate(str.maketrans('', '', string.punctuation))
    words = [w for w in text.split() if w.isalpha()]
    return len(words)
```

```
# Count pos and neg
```

```
pos_count = 0 if positive_review.strip() == "No Positive" else word_count(positive_review)
neg_count = 0 if negative_review.strip() == "No Negative" else word_count(negative_review)
```

```
print("Positive Word Count:", pos_count)
print("Negative Word Count:", neg_count)
```

```
# Create a DataFrame
```

```
new_data = pd.DataFrame([
    'overall_review': positive_review + " " + negative_review if negative_review != "No Negative" else positive_review,
    'Reviewer_Nationality': "United Kingdom",
    'Tags': "Leisure trip; Couple; Stayed 2 nights",
    'Hotel_Name': "Hotel ABC",
    'Review_Total_Positive_Word_Counts': pos_count,
    'Review_Total_Negative_Word_Counts': neg_count,
    'Total_Number_of_Reviews': 2850,
    'Total_Number_of_Reviews_Reviewer_Has_Given': 7,
    'days_since_review': 45.0
])
```

```
pipeline.predict(new_data)
print(f"Predicted Rating: {pred[0]:.2f}")
```

```
Positive Word Count: 14
Negative Word Count: 0
Predicted Rating: 9.40
```

✦ XGBoost & Gradient Boost

```
%pip install xgboost
```

```
Requirement already satisfied: xgboost in /usr/local/lib/python3.11/dist-packages (3.0.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from xgboost) (1.26.4)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.11/dist-packages (from xgboost) (2.23.4)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from xgboost) (1.13.1)
```

```
from sklearn.ensemble import GradientBoostingRegressor
from xgboost import XGBRegressor
```

```
# Compared with GBR and XGB model
models = {
    "GradientBoosting": GradientBoostingRegressor(random_state=42),
    "XGBoost": XGBRegressor(n_estimators=200, learning_rate=0.1, random_state=42, n_jobs=-1)
}

results = []
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

for name, model in models.items():
    pipeline = Pipeline(steps=[
        ('preprocessing', preprocessor),
        ('regressor', model)
    ])
    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)
    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    results.append({"Model": name, "MAE": round(mae, 3), "RMSE": round(rmse, 3)})

results_df = pd.DataFrame(results)
print(results_df)
```

	Model	MAE	RMSE
0	GradientBoosting	0.920	1.203
1	XGBoost	0.883	1.172

In this rating prediction task, XGBoost achieved the best performance (MAE = 0.883, RMSE = 1.172), outperforming both Random Forest (MAE = 0.901, RMSE = 1.239) and Gradient Boosting (MAE = 0.920, RMSE = 1.203). The results suggest that XGBoost captures complex feature interactions more effectively, leading to lower overall and extreme prediction errors, while RF and GBR still provided competitive baselines.

✦ Topic Modeling

To understand which aspects customers prioritize—such as location convenience, hotel decor, and other factors—I will perform topic modeling on the overall review texts. By combining this with word cloud visualizations, I aim to compare the results from both LDA and BERTopic methods to analyze topic differences and determine which approach produces more meaningful insights.

```
pip install pandas gensim nltk wordcloud matplotlib
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages (2.2.2)
Requirement already satisfied: gensim in /usr/local/lib/python3.11/dist-packages (4.3.3)
Requirement already satisfied: nltk in /usr/local/lib/python3.11/dist-packages (3.9.1)
Requirement already satisfied: wordcloud in /usr/local/lib/python3.11/dist-packages (1.9.4)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-packages (3.10.0)
Requirement already satisfied: numpy>=1.23.2 in /usr/local/lib/python3.11/dist-packages (from pandas) (1.26.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas) (2025.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas) (2025.2)
Requirement already satisfied: scipy<1.14.0,>=1.7.0 in /usr/local/lib/python3.11/dist-packages (from gensim) (1.13.1)
Requirement already satisfied: smart-open>=1.8.1 in /usr/local/lib/python3.11/dist-packages (from gensim) (7.3.0)
Requirement already satisfied: click in /usr/local/lib/python3.11/dist-packages (from nltk) (8.2.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.11/dist-packages (from nltk) (1.5.1)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.11/dist-packages (from nltk) (2024.11.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from nltk) (4.67.1)
Requirement already satisfied: pillow in /usr/local/lib/python3.11/dist-packages (from wordcloud) (11.3.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (1.3.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (4.55.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (1.4.7)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (25.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (3.2.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.11/dist-packages (from smart-open>=1.8.1->gensim) (3.13.1)
```

LDA

```
import pandas as pd
import gensim
from gensim import corpora
from gensim.models.ldamulticore import LdaMulticore
from gensim.models.coherencemodel import CoherenceModel
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
from nltk.corpus import stopwords
import nltk, string
```

```
nltk.download('stopwords')

# 0) Load original dataset
df = pd.read_csv("Hotel_Reviews.csv")

# 1) Build overall_review only if missing
if 'overall_review' not in df.columns:
    pos = df.get('Positive_Review', '').replace('No Positive', '').fillna('')
    neg = df.get('Negative_Review', '').replace('No Negative', '').fillna('')
    df['overall_review'] = (pos + ' ' + neg).str.strip()

# 2) Fast text prep
STOP = set(stopwords.words('english')).union(ENGLISH_STOP_WORDS)
PUNCT = str.maketrans('', '', string.punctuation)
tokens = (
    df['overall_review'].fillna('')
    .str.lower().str.translate(PUNCT).str.split()
    .apply(lambda xs: [w for w in xs if w.isalpha() and len(w) > 2 and w not in STOP])
)
tokens = tokens[tokens.map(len) > 0].reset_index(drop=True)

# 3) Dictionary/corpus with extremes filtering
dictionary = corpora.Dictionary(tokens)
dictionary.filter_extremes(no_below=10, no_above=0.5, keep_n=100_000)
corpus = [dictionary.doc2bow(t) for t in tokens]

# 4) Train LDA (multicore)
num_topics = 10
lda = LdaMulticore(
    corpus=corpus, id2word=dictionary, num_topics=num_topics,
    chunksize=4096, iterations=100, passes=1,
    alpha='asymmetric', eta='auto', random_state=42, workers=4
)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

```
print("\n🌟 Top LDA Topics:")
for tid, topic in lda.show_topics(num_topics=num_topics, num_words=8, formatted=False):
    words = [w for w, _ in topic]
    print(f"Topic {tid+1}: {' '.join(words)}")
```

```
🌟 Top LDA Topics:
Topic 1: location, staff, great, hotel, good, room, breakfast, friendly
Topic 2: hotel, room, location, station, good, close, walk, metro
Topic 3: room, hotel, small, bathroom, location, bed, staff, good
Topic 4: hotel, parking, room, booking, breakfast, pool, pay, car
Topic 5: hotel, room, check, breakfast, staff, bar, time, early
Topic 6: breakfast, bed, room, comfortable, staff, good, comfy, beds
Topic 7: room, view, floor, location, shower, good, bed, staff
Topic 8: staff, helpful, friendly, excellent, location, hotel, stay, breakfast
Topic 9: room, breakfast, coffee, good, tea, nice, bar, bathroom
Topic 10: staff, hotel, reception, room, rooms, like, service, night
```

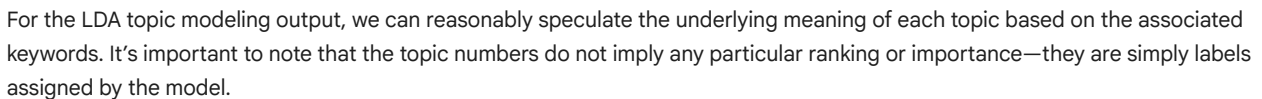
```
from wordcloud import WordCloud
import matplotlib.pyplot as plt

# Set topics number
num_topics = 10
cols = 5
rows = (num_topics + cols - 1) // cols # 自动计算行数

# Subplot
fig, axes = plt.subplots(rows, cols, figsize=(20, 8))
fig.suptitle("LDA Topic Word Clouds", fontsize=22, y=1.05)

# Iterate each word cloud
```

LDA Topic Word Clouds



- ```
from collections import Counter

Get each review's topic
dominant_topics = [
 max(lda.get_document_topics(bow), key=lambda x: x[1])[0]
 for bow in corpus
]

Calculate the number of occurrence for each topic
topic_counts = Counter(dominant_topics)

Print distribution of topics based on topic counts
print("\n📊 Topic Document Distribution:")
for topic_id, count in topic_counts.most_common():
 print(f"📌 Topic {topic_id+1}: {count} documents")
```

```

Topic Document Distribution:
Topic 1: 138157 documents
Topic 2: 68189 documents
Topic 8: 66222 documents
Topic 3: 59020 documents
Topic 6: 54000 documents
Topic 9: 28965 documents
Topic 5: 27010 documents
Topic 10: 25888 documents
Topic 4: 24756 documents
Topic 7: 19827 documents

```

To determine which topics dominate overall guest feedback, I calculated the frequency of each topic as the dominant one across all reviews. The results show that Topic 1 is the most prevalent, appearing in 138,157 documents, which suggests that guests prioritize location, staff, and overall experience the most. This is followed by Topic 2, related to accessibility and transportation, and Topic 8, which covers staff service and helpfulness.

Interestingly, this output differs from my initial assumption that room facilities or comfort would take the top spot. Instead, the findings indicate that location and service quality play the most crucial roles in shaping the overall guest experience. This insight can be highly valuable for hotel industry stakeholders when conducting market research or customer analysis, especially when planning to enter a competitive market.

## ▼ BERTopic

```
!pip install bertopic sentence-transformers hdbscan umap-learn
```

```

import pandas as pd
import numpy as np
from bertopic import BERTopic
from sklearn.feature_extraction.text import CountVectorizer
from collections import Counter
import matplotlib.pyplot as plt

```

[Show hidden output](#)

```

df = pd.read_csv("Hotel_Reviews.csv")
if 'overall_review' not in df.columns:
 pos = df.get('Positive_Review', '').replace('No Positive', '').fillna('')
 neg = df.get('Negative_Review', '').replace('No Negative', '').fillna('')
 df['overall_review'] = (pos.astype(str) + ' ' + neg.astype(str)).str.strip()

take sample from dataset
sample_n = 20_000
if sample_n is not None and len(df) > sample_n:
 df = df.sample(n=sample_n, random_state=42).reset_index(drop=True)

docs = df['overall_review'].fillna('').astype(str).tolist()

Vectorizer
vectorizer_model = CountVectorizer(
 stop_words="english",
 min_df=10,
 max_df=0.5
)

BERTopic
topic_model = BERTopic(
 vectorizer_model=vectorizer_model,
 language="english",
 calculate_probabilities=False,
 verbose=True,
)

topics, probs = topic_model.fit_transform(docs)

Print topics info
topic_info = topic_model.get_topic_info()
print("\nTop topics overview:")
print(topic_info.head(12))

```